

API Reference - JobbingTrack

Documentation complète des APIs REST de JobbingTrack v4.1.

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)

Vue d'ensemble

API REST moderne avec architecture microservices, authentification JWT et documentation OpenAPI.

Configuration de base

Base URL

```
Production: https://api.jobbingtrack.com
Développement: http://localhost:3000
```

Headers requis

```
Content-Type: application/json
Authorization: Bearer <jwt_token>
```

Authentification

```
# Obtenir un token
curl -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}'
```

Services API

Authentification

Base URL : `/auth` Port direct : `3001`

Connexion

```
POST /auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Inscription

```
POST /auth/register
Content-Type: application/json

{
  "email": "newuser@example.com",
  "password": "password123",
  "firstName": "John",
  "lastName": "Doe"
}
```

Vérification token

```
GET /auth/verify
Authorization: Bearer <token>
```

Candidatures

Base URL : /applications **Port direct :** 3002

Lister les candidatures

```
GET /applications?status=applied&limit=20&offset=0
Authorization: Bearer <token>
```

Créer une candidature

```
POST /applications
Authorization: Bearer <token>
Content-Type: application/json

{
  "companyId": "uuid",
  "position": "Développeur Full Stack",
  "description": "Mission passionnante...",
  "location": "Paris",
  "salaryMin": 45000,
  "salaryMax": 55000,
  "status": "applied"
}
```

Mettre à jour une candidature

```
PUT /applications/{id}
Authorization: Bearer <token>
Content-Type: application/json

{
  "status": "interview",
  "notes": "Entretien planifié"
}
```



Entreprises

Base URL : /companies **Port direct :** 3003

Lister les entreprises

```
GET /companies?industry=tech&size=50-200
Authorization: Bearer <token>
```

Créer une entreprise

```
POST /companies
Authorization: Bearer <token>
Content-Type: application/json

{
  "name": "TechCorp",
```

```
"website": "https://techcorp.com",  
"industry": "Technology",  
"size": "50-200",  
"description": "Entreprise innovante"  
}
```

Contacts

Base URL : /contacts **Port direct :** 3004

Lister les contacts

```
GET /contacts?companyId=uuid&search=john  
Authorization: Bearer <token>
```

Créer un contact

```
POST /contacts  
Authorization: Bearer <token>  
Content-Type: application/json  
  
{  
  "companyId": "uuid",  
  "firstName": "John",  
  "lastName": "Doe",  
  "email": "john.doe@techcorp.com",  
  "position": "CTO",  
  "phone": "+33123456789",  
  "linkedinUrl": "https://linkedin.com/in/johndoe"  
}
```

Entretiens

Base URL : /interviews **Port direct :** 3005

Lister les entretiens

```
GET /interviews?upcoming=true&date=2025-01-27  
Authorization: Bearer <token>
```

Planifier un entretien

```

POST /interviews
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "uuid",
  "contactIds": ["uuid1", "uuid2"],
  "scheduledAt": "2025-01-30T14:00:00Z",
  "type": "technical",
  "location": "Bureau Paris",
  "notes": "Entretien technique React/Node.js"
}

```

Appels

Base URL : /calls **Port direct :** 3006

Enregistrer un appel

```

POST /calls
Authorization: Bearer <token>
Content-Type: application/json

{
  "contactId": "uuid",
  "type": "outbound",
  "duration": 30,
  "notes": "Discussion sur le poste",
  "outcome": "interested"
}

```

Événements

Base URL : /events **Port direct :** 3007

Créer un événement

```

POST /events
Authorization: Bearer <token>
Content-Type: application/json

{
  "title": "Relance candidature",
  "description": "Appeler le recruteur",
}

```

```

"startDate": "2025-01-28T10:00:00Z",
"eventType": "reminder",
"entityType": "application",
"entityId": "uuid",
"reminderAt": "2025-01-28T09:00:00Z"
}

```

Suivi (Followups)

Base URL : /followups **Port direct :** 3008

Créer un suivi

```

POST /followups
Authorization: Bearer <token>
Content-Type: application/json

{
  "applicationId": "uuid",
  "contactId": "uuid",
  "type": "email",
  "scheduledAt": "2025-01-29T15:00:00Z",
  "notes": "Envoyer email de relance"
}

```

Profils

Base URL : /profiles **Port direct :** 3009

Mettre à jour le profil

```

PUT /profiles/me
Authorization: Bearer <token>
Content-Type: application/json

{
  "firstName": "John",
  "lastName": "Doe",
  "phone": "+33123456789",
  "preferences": {
    "theme": "dark",
    "language": "fr",
    "notifications": {
      "email": true,
      "push": true,
      "reminders": true
    }
  }
}

```

```
}  
}  
}
```

Notifications

Base URL : /notifications **Port direct :** 3010

Lister les notifications

```
GET /notifications?unread=true&limit=10  
Authorization: Bearer <token>
```

Marquer comme lue

```
PUT /notifications/{id}/read  
Authorization: Bearer <token>
```

Dashboard

Base URL : /dashboard **Port direct :** 3012

Vue d'ensemble

```
GET /dashboard/overview  
Authorization: Bearer <token>
```

Analytics

```
GET /dashboard/analytics?period=30d&metrics=applications,interviews  
Authorization: Bearer <token>
```

Codes d'erreur

Format des erreurs

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Données invalides",
    "details": {
      "email": ["Champ requis", "Format invalide"]
    }
  },
  "timestamp": "2025-01-27T10:00:00Z"
}
```

Codes d'erreur courants

Code	HTTP Status	Description
VALIDATION_ERROR	400	Données invalides
UNAUTHORIZED	401	Authentification requise
FORBIDDEN	403	Permissions insuffisantes
NOT_FOUND	404	Ressource introuvable
CONFLICT	409	Conflit de données
RATE_LIMITED	429	Limite de requêtes atteinte
INTERNAL_ERROR	500	Erreur serveur



Authentification

JWT Tokens

- **Access Token** : 1 heure de validité
- **Refresh Token** : 30 jours de validité
- **Format** : Bearer <token>

Rôles et permissions

```
{
  "user": {
    "permissions": ["read", "write_own"]
  }
}
```



```

},
"admin": {
  "permissions": ["read", "write", "delete", "manage_users"]
},
"super_admin": {
  "permissions": ["read", "write", "delete", "manage_users", "system_admin"]
}
}

```

Rate Limiting

- **Général** : 1000 requêtes/15min
- **Authentification** : 5 tentatives/5min
- **Endpoints sensibles** : 10 requêtes/min



Exemples d'utilisation

Création complète d'une candidature

```

// 1. Authentification
const loginResponse = await fetch('/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    email: 'user@example.com',
    password: 'password123'
  })
});

const { accessToken } = await loginResponse.json();

// 2. Création d'une entreprise
const companyResponse = await fetch('/companies', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    name: 'TechCorp',
    website: 'https://techcorp.com',
    industry: 'Technology'
  })
});

const { data: company } = await companyResponse.json();

```

```
// 3. Création d'un contact
const contactResponse = await fetch('/contacts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    companyId: company.id,
    firstName: 'Jane',
    lastName: 'Smith',
    email: 'jane.smith@techcorp.com',
    position: 'HR Manager'
  })
});

const { data: contact } = await contactResponse.json();

// 4. Création de la candidature
const applicationResponse = await fetch('/applications', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    companyId: company.id,
    position: 'Développeur Full Stack',
    status: 'applied',
    notes: 'Candidature spontanée'
  })
});

const { data: application } = await applicationResponse.json();

// 5. Planification d'un entretien
await fetch('/interviews', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({
    applicationId: application.id,
    contactIds: [contact.id],
    scheduledAt: '2025-02-01T14:00:00Z',
    type: 'technical'
  })
});
```

Utilisation avec cURL

```
# Authentification
TOKEN=$(curl -s -X POST http://localhost:3000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"user@example.com","password":"password123"}' \
  | jq -r '.data.accessToken')

# Lister les candidatures
curl -X GET "http://localhost:3000/applications" \
  -H "Authorization: Bearer $TOKEN" \
  | jq '.'

# Créer une entreprise
curl -X POST http://localhost:3000/companies \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{
    "name": "TechCorp",
    "industry": "Technology",
    "website": "https://techcorp.com"
  }' \
  | jq '.'
```



Ressources

- [Architecture Microservices](#) - Vue technique complète
- [Base de Données](#) - Schémas PostgreSQL et relations
- [Services Détaillés](#) - Détail de tous les microservices
- [Endpoints API](#) - Liste complète des endpoints
- [Guide de Déploiement](#) - Installation et configuration
- [Guide de Développement](#) - Configuration environnement

Version: 4.1 - API étendue **Dernière mise à jour:** Octobre 2025